

B4 — ORDONNANCEMENT INDUSTRIEL

Resource-Constrained Project Scheduling Problem

Modélisation CP-SAT · Algorithme Génétique (PPX) · Benchmark PSPLIB

Arthur Gallier · Nicolas Naegelen

PLAN

Plan de la présentation

01

Problème RCPSP

Définition, formulation, complexité

02

Contexte & littérature

Histoire, classes d'algorithmes

03

Modèle exact CP-SAT

Variables, contraintes, objectif

04

Métaheuristique GA

Population, SGS, croisement PPX

05

Benchmark PSPLIB

j30 / j60 / j120, comparaison BKS

06

Stratégies de branchement

AUTOMATIC vs LP vs PseudoCost

07

RCPSP/max

Contraintes temporelles généralisées

08

RCPSP multi-mode

Intervalles optionnels, AddExactlyOne

09

Résultats & perspectives

Synthèse, limites, ouvertures

Problème RCPSP

FORMULATION

- n tâches avec durées p_j , reliées par contraintes de précédence
 - › Graphe orienté acyclique (AON — Activity-On-Node)
- R ressources renouvelables, chacune de capacité R_r
 - › Demande cumulée à chaque instant $t \leq R_r$
- Objectif : minimiser le makespan $C_{\max}$
 - › Supersource & supersink virtuels (durée 0) encadrent le réseau
- NP-difficile au sens fort
 - › Exact tractable $\leq \sim 60$ tâches ; au-delà, métaheuristiques

FORMULATION MATHÉMATIQUE

$\min \quad C_{\max} = \text{start}[n] + \text{dur}[n]$

s.t. $\text{start}[j] \geq \text{end}[i]$
 $\forall (i,j) \in E$
 $\sum \text{req}[j,r] \leq \text{cap}[r]$
 $\forall r, t$
 $\text{start}[j] \in \mathbb{N}$

BENCHMARK PSPLIB

j30 480 instances · 30 tâches

j60 480 instances · 60 tâches

j120 600 instances · 120 tâches

Format .sm — parseur dédié

Contexte historique & classes d'algorithmes

ÉVOLUTION

- 1963** ● **Kelley & Walker**
Critical Path Method (CPM) — ordonnancement sans ressources
- 1975** ● **Pritsker et al.**
Première formulation RCPSP — Branch & Bound exact
- 1998** ● **Kolisch & Hartmann**
Survey benchmark PSPLIB — PPX crossover pour GA
- 2003** ● **Baptiste & Nuijten**
Propagation par raisonnement énergétique (OR)
- 2011** ● **Google OR-Tools**
CP-SAT : CDCL(T) + LP relaxation + portfolio
- 2021** ● **État de l'art**
+ RCPSP/max, multi-mode, stochastique, online...

CLASSES D'ALGORITHMES

Méthodes exactes

- › Branch & Bound
- › CP / IP / MIP

Heuristiques constructives

- › SGS série / parallèle
- › Priorité (LFT, LST, ...)

Métaheuristiques

- › Algorithmes génétiques
- › Recuit simulé, Tabou
- › ACO, PSO

Hybrides

- › LNS + CP, Lagrangien

CP-SAT (OR-Tools)

Contraintes cumulatives & IntervalVar

ÉLÉMENTS DU MODÈLE

- Variables : $\text{start}[j] \in [0, H]$, $\text{end}[j] = \text{start}[j] + \text{dur}[j]$
- IntervalVar encapsule (start, durée, end) — nécessaire à AddCumulative
- Précédences : `model.Add(start[s] ≥ end[j])`
- Ressources : `model.AddCumulative(intervals, demands, capacity)`
- Objectif : `model.Minimize(end[supersink])`

```
for j in range(1, jobs+1):
    s = model.NewIntVar(0, H, f's{j}')
    e = model.NewIntVar(0, H, f'e{j}')
    t = model.NewIntervalVar(s, dur[j], e)

for j, succs in prec.items():
    for k in succs:
        model.Add(starts[k] >= ends[j])

for r in range(num_r):
    model.AddCumulative(ivs_r, dems_r, cap[r])
model.Minimize(ends[supersink])
```

COMMENT FONCTIONNE CP-SAT ?

- CDCL(T) — Conflict-Driven Clause Learning
 - › Extension aux domaines finis (entiers)
- LP Relaxation à chaque nœud
 - › Bornes duales fortes
- Propagateurs spécialisés
 - › AddCumulative → timetable + edge-finding
 - › AddNoOverlap → disjonctif
- Portfolio de stratégies en parallèle
 - › Workers indépendants, partage de clauses

Algorithme Génétique (PPX)

D'après Hartmann (1998)

MÉCANIQUE DE L'ALGORITHME

- Représentation : permutation topologique de tâches
- Fitness : Serial SGS — planifie chaque tâche au plus tôt
- Sélection : élitisme, top 25 % conservés
- Croisement PPX — Precedence Preserving Crossover
 - › Masque binaire ; préserve toujours la topologie

```
def ppx_crossover(p1, p2):  
    mask = [randint(0,1) for _ in range(n)]  
    child = []  
    for bit in mask:  
        if bit == 0:  
            job = next_from(p1)  
        else:  
            job = next_from(p2)  
        child.append(job)  
    return child    # toujours valide
```

BOUCLE D'ÉVOLUTION

- 1 Initialisation**
pop = 50
- 2 Évaluation SGS**
→ makespan
- 3 Élitisme**
top 25 %
- 4 PPX Crossover**
× 50 %
- 5 Remplissage**
aléatoire
- 6 Itérer**
× 100 générations

Benchmark PSPLIB

CP-SAT (exact) vs GA (PPX) — temps limité

Dataset	Tâches	CP makespan	Statut	GA makespan	Gap GA/CP	Temps CP	Temps GA
j30	30	OPTIMAL	≤ 12 s	$\approx$ optimal	2–7 %	~ 0 s	~ 0.2 s
j60	60	OPTIMAL	≤ 20 s	$\approx$ optimal	0–10 %	~ 0 s	~ 0.3 s
j120	120	OPTIMAL	≤ 30 s	$\approx$ optimal	6–7 %	~ 0 s	~ 0.5 s

OBSERVATIONS CLÉS

- CP-SAT atteint l'optimal sur toutes les instances en < 1 s (Set 10)
 - › Instances simples ; les cas difficiles sont dans d'autres sets à forte densité
- GA (PPX, 60 gén.) dévie de 0–10 % de l'optimal — bon trade-off qualité/temps
 - › Scalabilité quasi-linéaire : 0.2 s $\rightarrow$ 0.5 s pour 4× plus de tâches
- Limitation GA : sans mutation explicite, risque de convergence prématurée

search_branching

Impact sur l'exploration de l'arbre CP-SAT

Stratégie	ID	Description
AUTOMATIC_SEARCH	0	Portfolio interne adaptatif (défaut)
PORTFOLIO_SEARCH	2	Combinaison de plusieurs heuristiques
LP_SEARCH	3	Branchement guidé par la relaxation LP
PSEUDO_COST_SEARCH	4	Coût pseudo-unitaire — analogue SAT VSIDS
PORTFOLIO_WITH_QUICK_RESTART	5	Portfolio + redémarrages rapides

RCPSP/max

Contraintes temporelles généralisées

GÉNÉRALISATION

- Précédence généralisée : $\text{start}[j] - \text{start}[i] \geq \delta_{ij}$
 - › Standard : $\delta_{ij} = \text{dur}[i] \Rightarrow \text{start}[j] \geq \text{end}[i]$
 - › Lag positif : $\delta_{ij} > \text{dur}[i]$ — délai minimum
 - › Lag négatif : $\delta_{ij} < 0$ — chevauchement autorisé
- Une seule contrainte CP-SAT supplémentaire suffit
- Applications : montage, BTP, séchage industriel

MISE EN ŒUVRE CP-SAT

```
for (i, j, delta) in temporal:
    model.Add(
        starts[j] - starts[i] >= delta)

# (2, 3, -2) : start[3] >= start[2] - 2
# tâche 3 peut débiter 2u
# avant la fin de 2
```

RÉSULTAT (7 TÂCHES)

−10 %

makespan réduit grâce au lag négatif (de 10 → 9)

RCPSP multi-mode

NewOptionalIntervalVar + AddExactlyOne

PRINCIPE

- Chaque tâche j dispose de M_j modes : (durée, demandes)
 - › Mode rapide : durée courte, forte demande
 - › Mode lent : durée longue, faible demande
- Un BoolVar par (tâche, mode)
 - › AddExactlyOne(mode_vars[j]) — exactement un mode actif
 - › NewOptionalIntervalVar activé si BoolVar = True
- AddCumulative ne compte que les intervalles actifs

CODE CP-SAT

```
for k, (dur, reqs) in
    enumerate(modes[j]):
    mv = model.NewBoolVar(
        f'm{j}_{k}')
    iv = model.NewOptional
        IntervalVar(
            starts[j], dur,
            ends[j], mv)

    model.AddExactlyOne(
        mode_vars[j])
```

RÉSULTAT (6 TÂCHES, 2 MODES)

CP-SAT sélectionne automatiquement le mode optimal sous contrainte de ressource.

Résultats, limites & perspectives

SYNTHÈSE

- CP-SAT : optimal sur j30–j120 Set 10 en < 1 s
- GA (PPX) : gap moyen 3–7 % sur j30, scalable
- RCPSP/max : lag négatif $\rightarrow$ makespan -10 %
- Multi-mode : sélection auto du mode optimal
- Branchement : AUTOMATIC $\approx$ PORTFOLIO en pratique

LIMITES

- GA sans mutation : risque de convergence prématurée
 - › Solution : ajouter mutation par swap de gènes
- BKS j30 Set 10 non disponibles publiquement
- Instances Set 10 trop faciles pour CP-SAT
 - › Tester sur instances haute densité ($rf = 1.0$)
- Multi-mode : pas de benchmark public dédié testé

PERSPECTIVES

- Large Neighborhood Search (LNS)
 - › Hybridation CP + méta : ré-optimiser sous-ensemble
- RCPSP stochastique
 - › Durées aléatoires $\rightarrow$ politiques robustes
- RCPSP online / temps réel
 - › Pannes ressources, replanification
- Reinforcement Learning
 - › Politique de dispatch apprise

Merci pour votre attention.

Questions ?
